

《算法设计技巧与分析》

第3章 堆

朱友文

zhuyw@nuaa.edu.cn

为什么需要“堆”？

- 在许多场景中，我们需要一个数据结构存储一些元素
- 并需要大量用到如下两种操作：**插入元素**和**寻找最大(小)值元素**。
- 为了提高这两种运算的效率，使用什么数据结构合适？

为什么需要“堆”？

- 大量用到2种操作：**插入元素**和**寻找最大(小)值元素**。

– **普通数组**：易插入元素，但求最大(小)值元素需要搜索整个队列。

插入元素： $O(1)$ ；求最大元素： $O(n)$

– **有序数组**：易找到最大(小)值，但插入元素需要移动大量元素。

插入元素： $O(n)$ ；求最大元素： $O(1)$

为什么需要“堆”？

- 大量用到2种操作：插入元素和寻找最大(小)值元素。

– 堆则是一种有效实现上述两种运算的简单数据结构。

插入元素： $O(\log n)$ ； 求最大元素： $O(1)$

什么是“堆”？

➤堆的定义（2个条件）

- 条件1：堆是几乎完全二叉树
- 条件2：堆中任何节点 v ， v 的父节点中元素值不小于 v 的元素值（大顶堆）

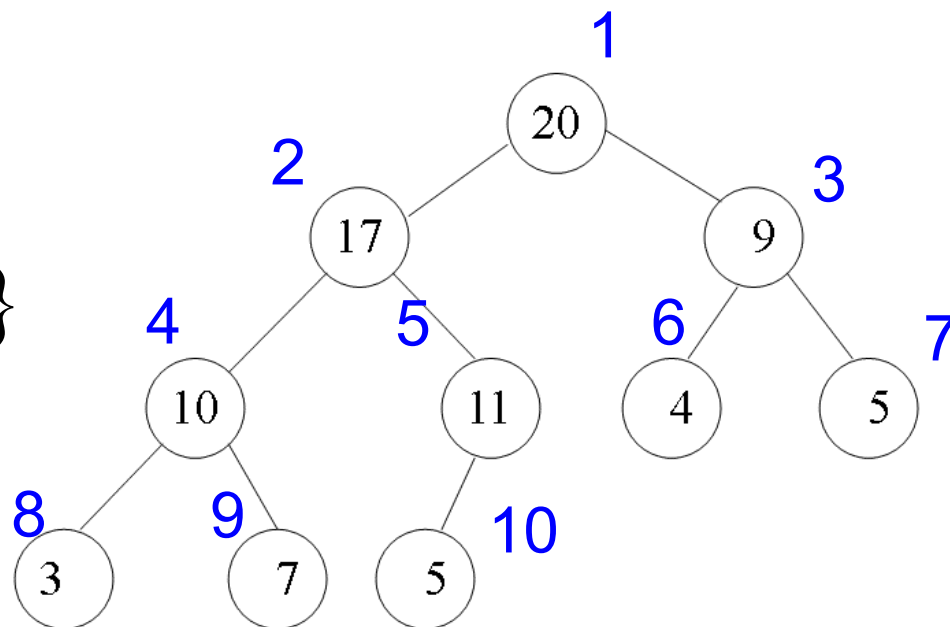
➤可以类似地定义小顶堆。这里，我们以大顶堆为准进行分析

什么是“堆”？

➤ 存储方式

➤ 数组 $H[1,2,\dots,n]$

➤ 表示方法： $\{H, n\}$



20	17	9	10	11	4	5	3	7	5
1	2	3	4	5	6	7	8	9	10

什么是“堆”？

➤ 根节点 $H[1]$

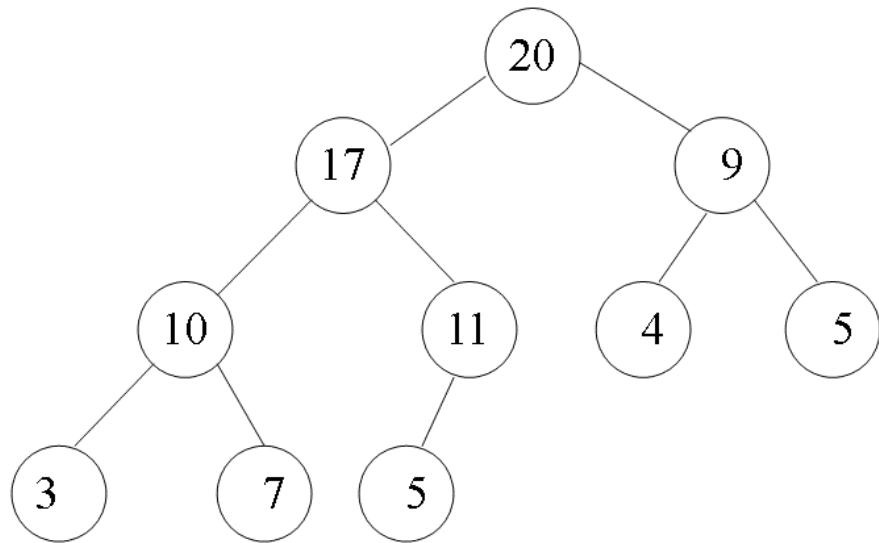
➤ $H[j]$ 的左右孩子节点是？

左孩子： $H[2*j]$ ； 右孩子： $H[2*j+1]$

➤ 如果 $j > 1$, $H[j]$ 的父节点是 $H[\lfloor j/2 \rfloor]$

➤ 如果堆 H 有 n 个节点，那么 H 对应的几乎完全二叉树的高度是 $\lfloor \log n \rfloor$

什么是“堆”？



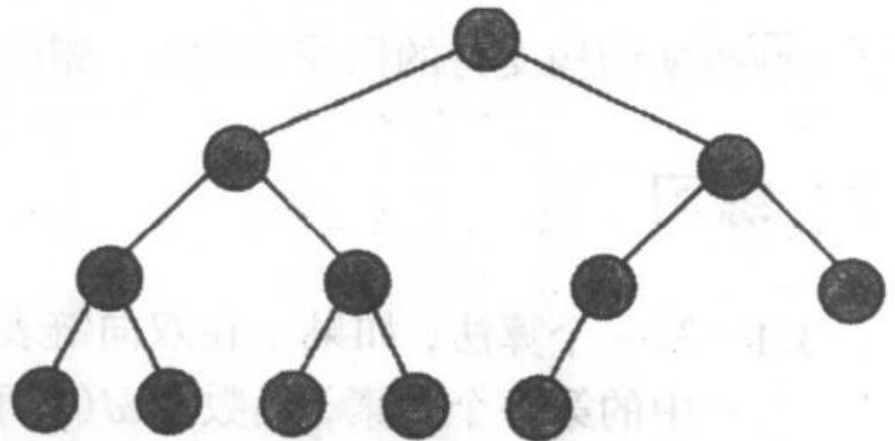
观察结论：

- 根节点键值最大，叶子节点键值较小。从根到叶子，键值以非升序排列。
- 节点的左右儿子节点键值并无顺序要求。
- 堆的数组表示呈“基本有序”状态。但是，并非节点的高度越高，键值就越大。

什么是“堆”？

假设 $H[1,2,\dots,n]$ 是一个大顶堆，那么，

- 第二大元素可能存在于哪些位置？
- 第三大元素可能存在于哪些位置？
- 最小元素可能存在于哪个位置？
- 求最小元素的时间复杂度是？



什么是“堆”？

下面数组中哪一个是堆？

(a)

8	6	4	3	2
---	---	---	---	---

(c)

9	7	5	6	3
---	---	---	---	---

(e)

9	4	7	2	1	6	5	3
---	---	---	---	---	---	---	---

(b)

7

(d)

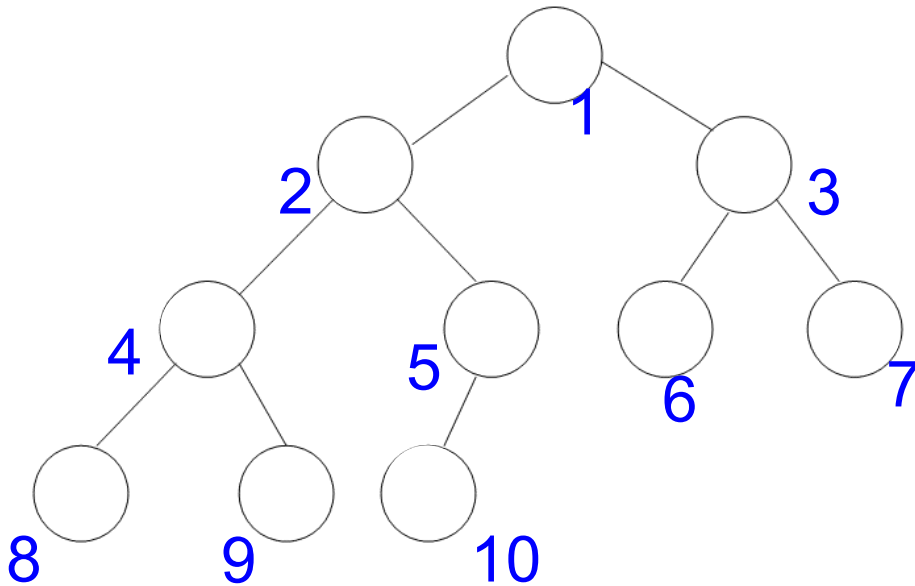
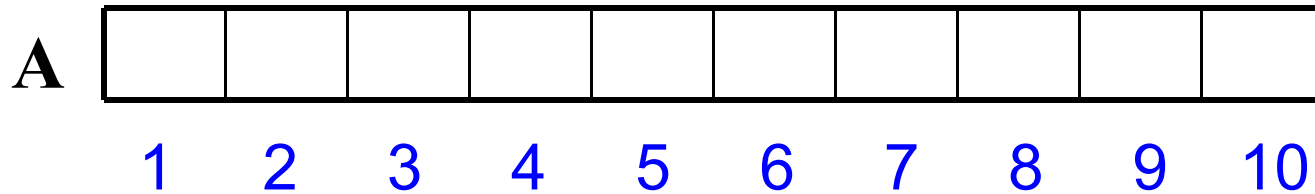
9	4	8	3	2	5	7
---	---	---	---	---	---	---

你是怎么判断每个数组是否是“堆”的？

给定数组 $A[1,2,\dots,n]$ ，如何判断它是否是堆？ 给出算法

什么是“堆”？

给定数组 $A[1,2,\dots,n]$, 如何判断它是否是堆? 给出算法



HeapTest(H, n)

输入：数组 $H[1 \dots n]$,

输出：若 H 满足堆特性返回1，否则返回0

for($i = \lfloor n/2 \rfloor$; $i \geq 1$; $i--$) {

$j = 2i$;

if(($j+1 \leq n$) and $H[j+1] > H[j]$) **then** $j = j+1$; //取左右孩子中较大者

if ($H[i] < (H[j])$) **return** 0; //不满足堆特性，返回0

}

return 1;

“堆” 上的基本操作

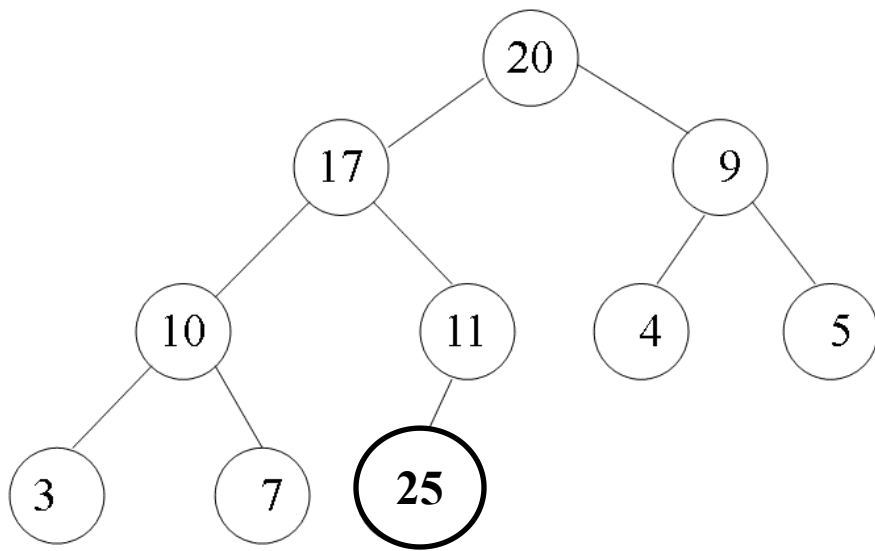
- `make-heap(A)`: 从数组A创建堆
- `insert(H, x)`: 插入元素 x 到堆H中
- `delete(H, i)`: 删除堆H的第 i 项 $H[i]$
- `delete-max(H)`: 从非空堆H中删除最大键值并返回数据项

如何实现?

堆上的辅助运算

➤ Sift-up

- 若某个节点 $H[i]$ 键值大于其父节点的键值，就违背了堆的特性，需要进行调整。

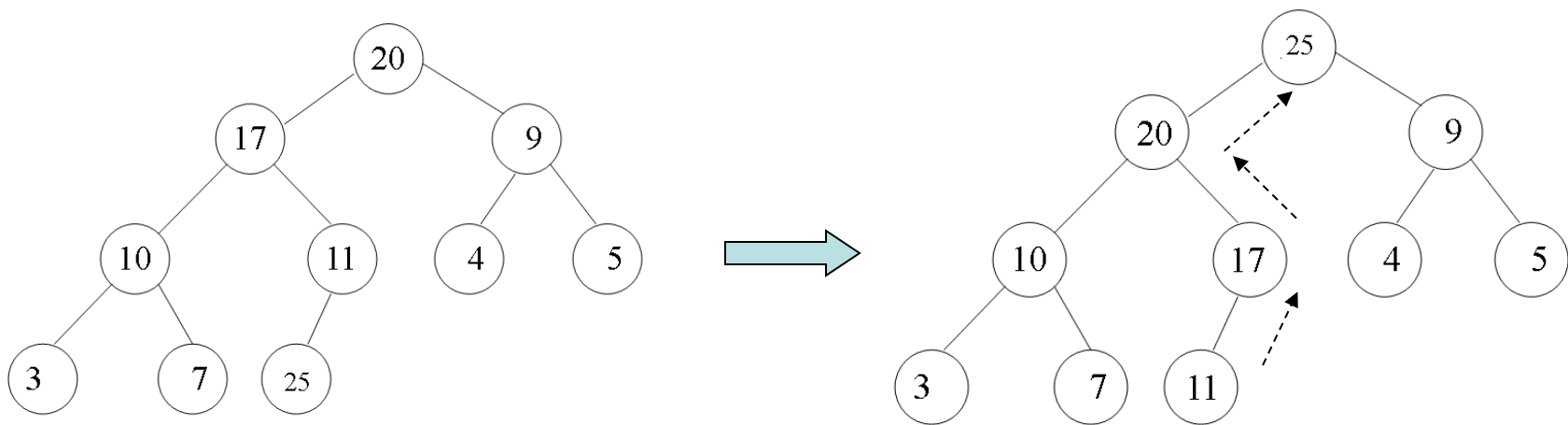


- 调整方法：上移

堆上的辅助运算

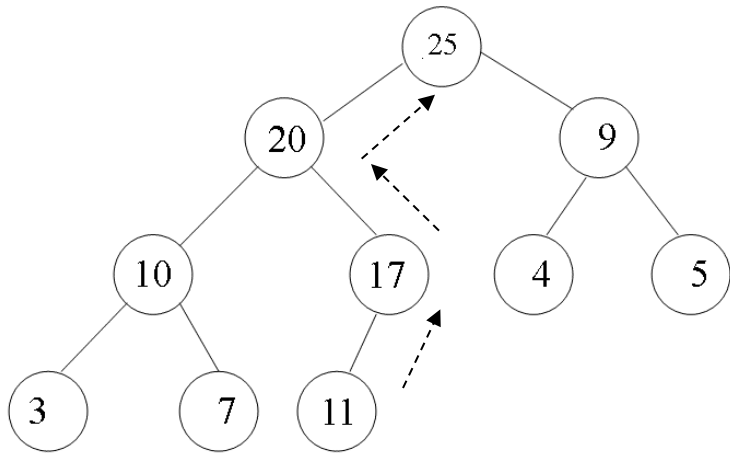
➤ Sift-up

- 若某个节点 $H[i]$ 键值大于其父节点的键值，就违背了堆的特性，需要进行调整。调整方法：上移



堆上的辅助运算

➤ Sift-up



```
Sift-up(H, i)
{
  while (i > 1) {
    if (H[i] > H[⌊i/2⌋]) 互换H[i]和H[⌊i/2⌋];
    else return; //调整过程至此已满足要求, 可退出

    i ← ⌊i/2⌋;
  } //调整进行到根节点, //或到某一节点终止
  return;
}
```

堆上的辅助运算

Sift-up(H, i)

输入: 数组 $H[1...n]$, 索引 $1 \leq i \leq n$

输出: 上移 $H[i]$ (如果需要), 使它的键值不大于父节点的键值
{

while ($i > 1$) {

if ($H[i] > H[\lfloor i/2 \rfloor]$) **then** 互换 $H[i]$ 和 $H[\lfloor i/2 \rfloor]$;

else return; //调整过程至此已经满足要求, 可退出

$i \leftarrow \lfloor i/2 \rfloor$;

} //调整进行到根节点, 或到某一节点终止

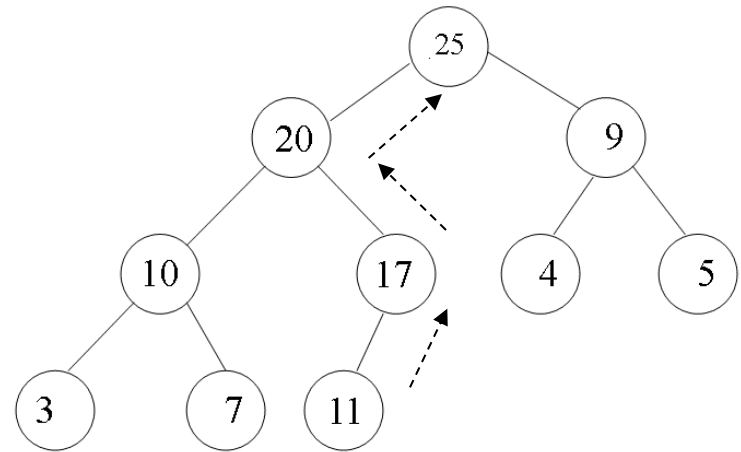
return;

}

堆上的辅助运算

➤ Sift-up

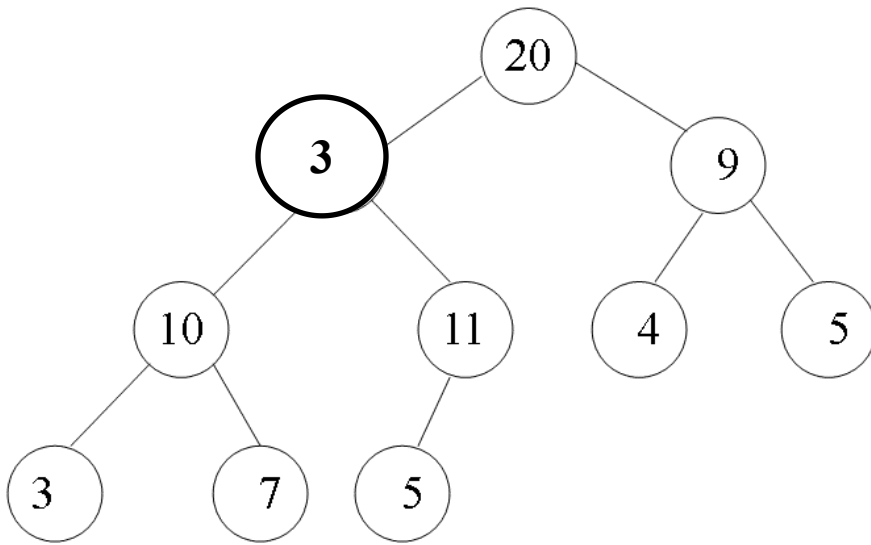
- 时间复杂度: $O(\log n)$
- 空间复杂度: $O(1)$



堆上的辅助运算

➤ Sift-down

- 假如某个内部节点 $H[i]$ ($i \leq \lfloor n/2 \rfloor$), 其键值小于儿子节点的键值, 即 $\text{key}(H[i]) < \text{key}(H[2i])$ 或 $\text{key}(H[i]) < \text{key}(H[2i+1])$ (如果右儿子存在), 违背了堆特性, 需要进行调整。

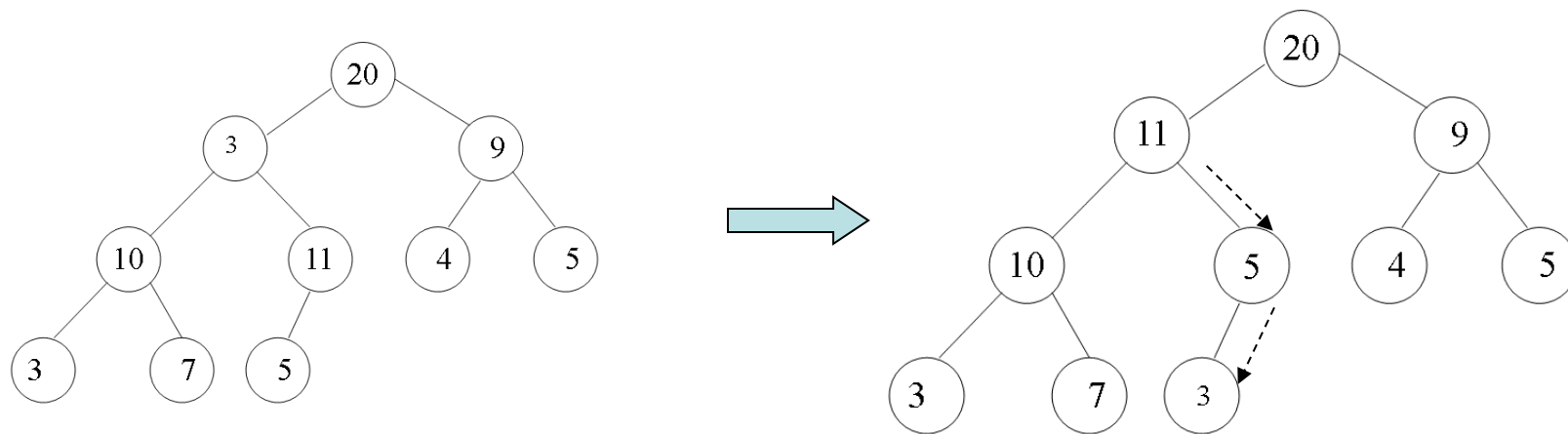


- 调整方法: 下移

堆上的辅助运算

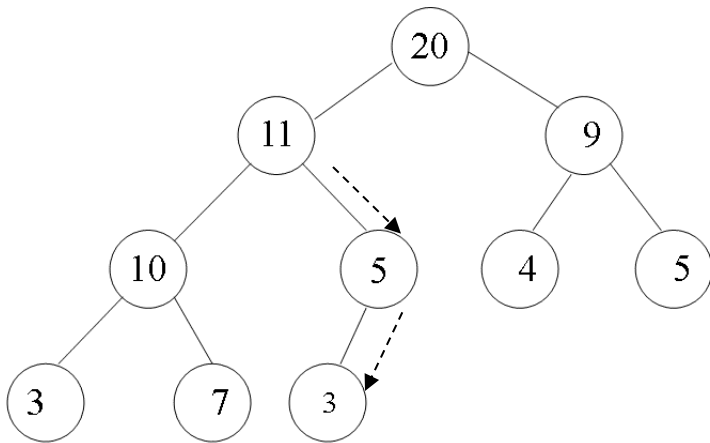
➤ Sift-down

- 假如某个内部节点 $H[i]$ ($i \leq \lfloor n/2 \rfloor$), 其键值小于儿子节点的键值, 即 $\text{key}(H[i]) < \text{key}(H[2i])$ 或 $\text{key}(H[i]) < \text{key}(H[2i+1])$ (如果右儿子存在), 违背了堆特性, 需要进行调整。调整方法: 下移



堆上的辅助运算

➤ Sift-down



Sift-down(H,i)

{

while($2i \leq n$) {

$i = 2i$;

if(($i+1 \leq n$) and $H[i+1] > H[i]$) **then** $i = i+1$;

//取左右孩子中较大者

if ($H[\lfloor i/2 \rfloor] < (H[i])$) 互换 $H[i]$ 和 $H[\lfloor i/2 \rfloor]$;

else return;

//调整至此已满足堆特性, 可退出

end if

}

return;

}

堆上的辅助运算

Sift-down(H, i)

输入：数组 $H[1 \dots n]$ ，索引 $1 \leq i \leq n$

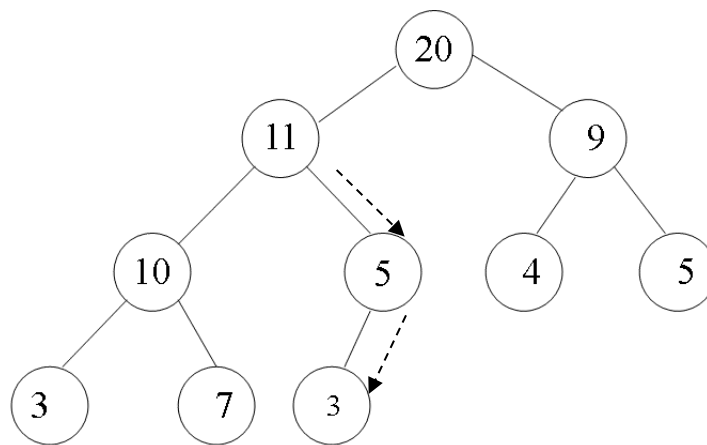
输出：下渗 $H[i]$ (若它违背了堆特性)，使 H 满足堆特性

```
{  
while( $2i \leq n$ ) {  
     $i = 2i$ ;  
    if( $(i+1 \leq n)$  and  $H[i+1] > H[i]$ ) then  $i = i+1$ ; //取左右孩子中较大者  
    if ( $H[\lfloor i/2 \rfloor] < (H[i])$ ) then 互换  $H[i]$  和  $H[\lfloor i/2 \rfloor]$ ;  
    else return; //调整过程至此已经满足堆特性，可退出  
    end if  
}  
return;  
}
```

堆上的辅助运算

➤ Sift-down

- 时间复杂度: $O(\log n)$
- 空间复杂度: $O(1)$



堆上的辅助运算

- Sift-up和Sift-down的适用条件：
除了 $H[i]$ ，都满足堆的要求

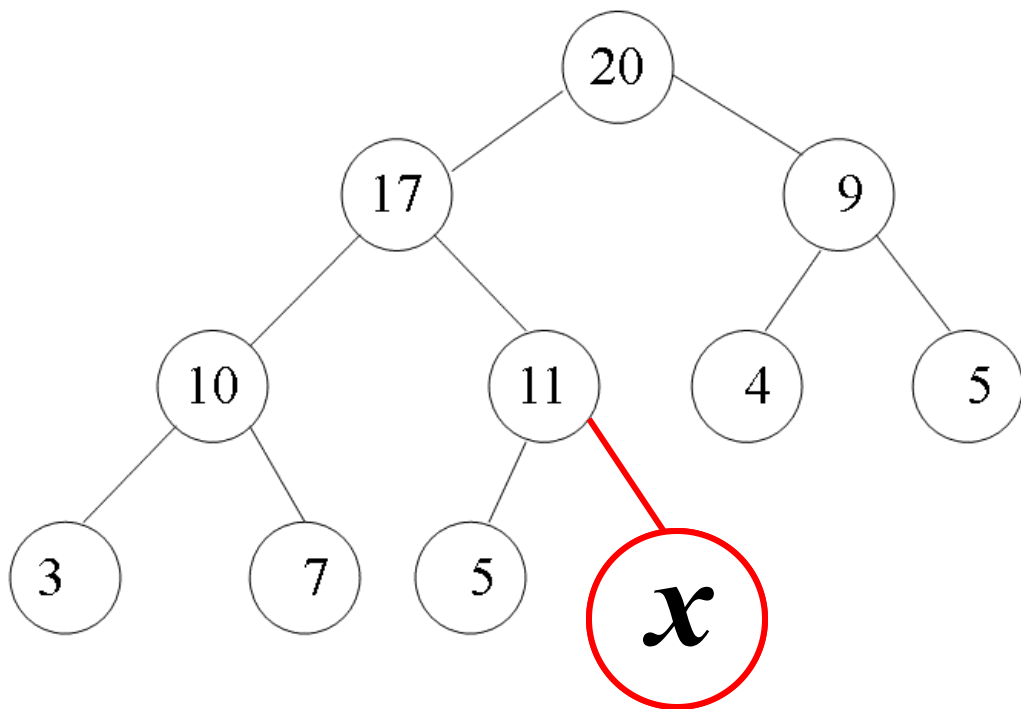
“堆” 上的基本操作

- `make-heap(A)`: 从数组A创建堆
- `insert(H, x)`: 插入元素 x 到堆H中
- `delete(H, i)`: 删除堆H的第 i 项 $H[i]$
- `delete-max(H)`: 从非空堆H中删除最大键值并返回数据项

如何实现?

堆上的基本运算

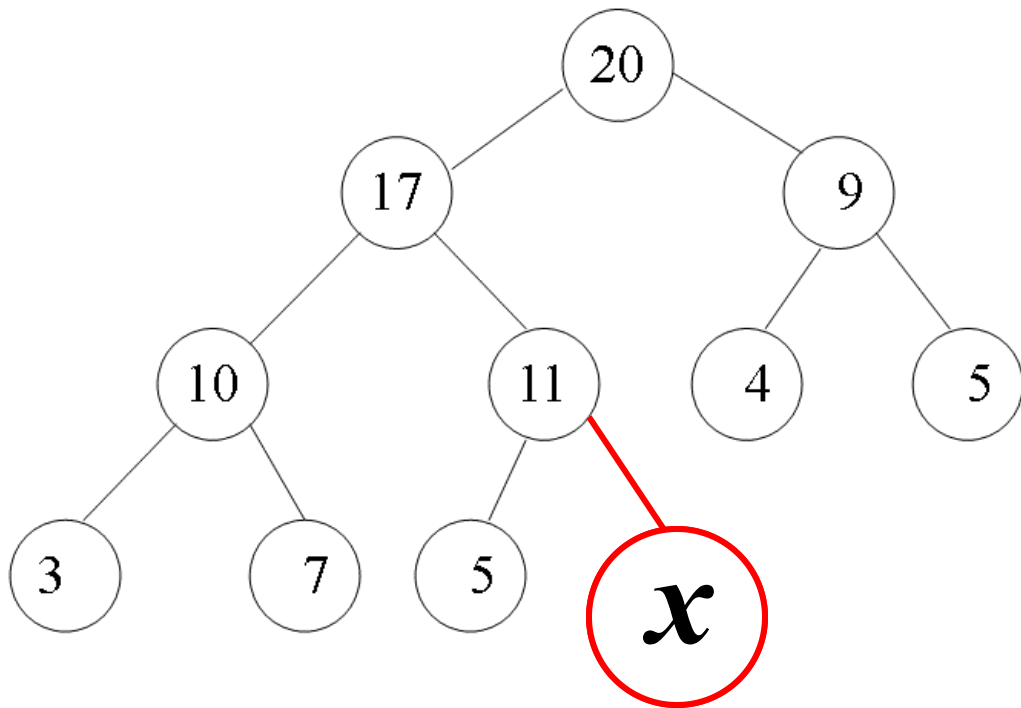
➤ 插入: $\text{Insert}(H, x)$



- H的长度加一
- 新加的最后一个元素值设为 x
- Sift-up最后一个元素

堆上的基本运算

➤ 插入: $\text{Insert}(H, x)$



- H 的长度加一
- 新加的最后一个元素值设为 x
- Sift-up 最后一个元素

$n = n + 1;$

$H[n] = x;$

$\text{Sift-Up}(H, n);$

堆上的基本运算

➤ 插入: $\text{Insert}(H, x)$

输入: 堆 $H[1 \cdots n]$ 和元素 x 。

输出: 新的堆 $H[1 \cdots n+1]$, x 为其元素之一。

$\{$ $n = n+1;$
 $H[n] = x;$
 $\text{Sift-Up}(H, n);$
 $\}$

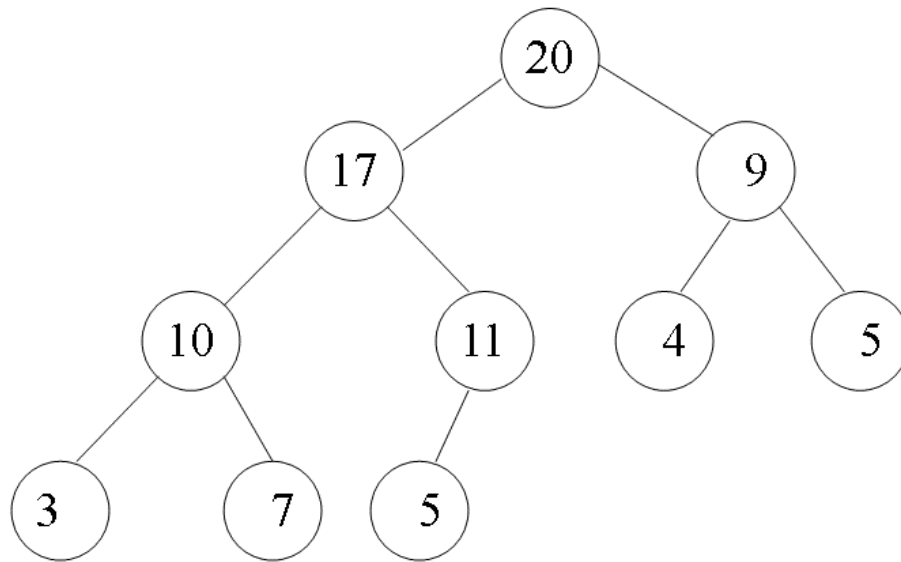
堆上的基本运算

➤ 插入: $\text{Insert}(H, x)$

- 时间复杂度: $O(\log n)$
- 空间复杂度: $O(1)$

堆上的基本运算

- 删除: $\text{Delete}(H, i)$ // 假设堆为 $\{H, n\}$
- 先看一个特殊情况: 如果 $i=n$, 即删除最后一个元素, 怎么实现? 方法: $n--$

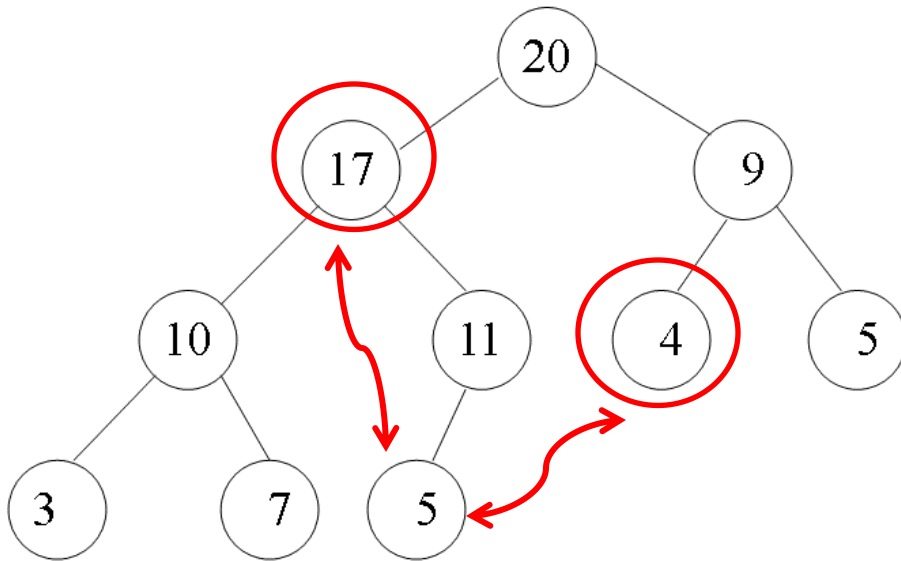


堆上的基本运算

➤ 删除: $\text{Delete}(H, i)$ // 假设堆为 $\{H, n\}$

➤ 如果 $i < n$ 呢?

- 互换 $H[i]$ 和 $H[n]$,
- H 的长度减一,
- 如 $H[i] < H[n]$,
 $\text{Sift-up}(H, i)$;
- 否则 $\text{Sift-down}(H, i)$



堆上的基本运算

➤ 删除: $\text{Delete}(H, i)$

输入: 非空堆 $H[1 \cdots n]$ 和位于 1 和 n 之间的索引 i 。

输出: 删除 $H[i]$ 之后的新堆 $H[1 \cdots n - 1]$ 。

```
{  $x = H[i]; y = H[n];$   
   $n--;$   
  if( $i = n + 1$ ) return;  
   $H[i] = y;$   
  if( $y > x$ ) Sift-Up( $H, i$ );  
  else Sift-Down( $H, i$ );  
  return; }
```

堆上的基本运算

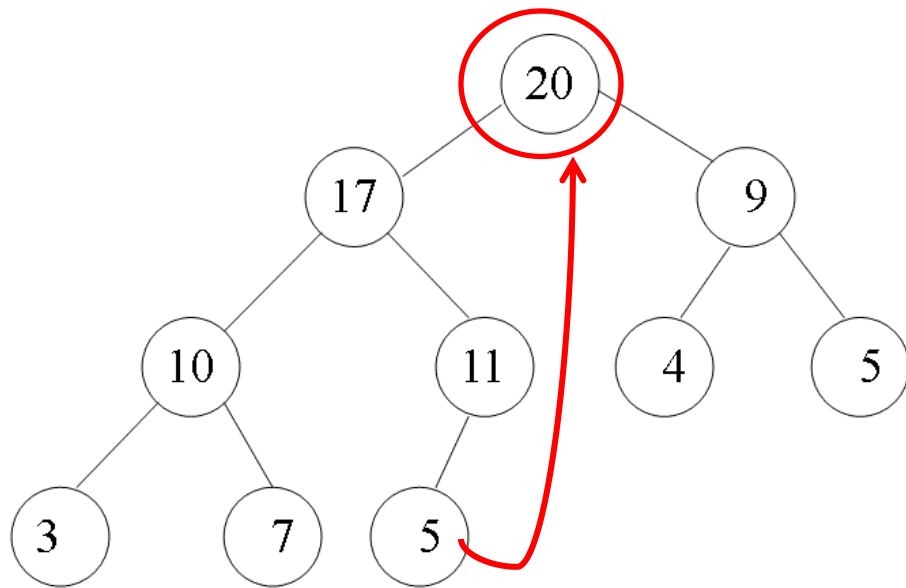
➤ 插入: $\text{Delete}(H, i)$

- 时间复杂度: $O(\log n)$
- 空间复杂度: $O(1)$

堆上的基本运算

➤ 删除最大值: $\text{Delete-Max}(H)$

最大值存储在哪个位置?



堆上的基本运算

➤ 删除最大值: $\text{Delete-Max}(H)$

输入: 堆 $H[1..n]$

输出: 返回最大键值元素, 并将其从堆中删除

{

$x = H[1];$

$H[1] = H[n];$

$n--;$

$\text{Sift-Down}(H, 1);$

$\text{return } x;$

}

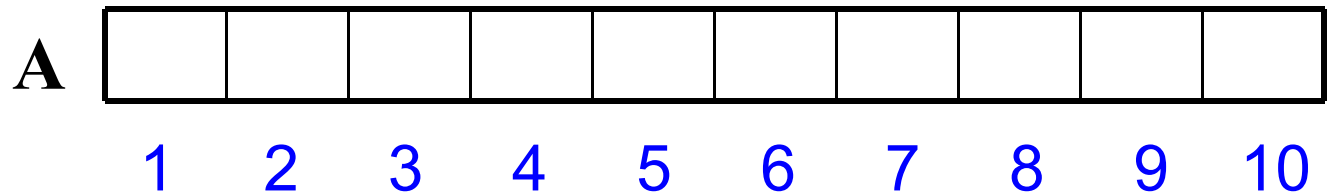
堆上的基本运算

➤ 插入: $\text{Delete-max}(H)$

- 时间复杂度: $O(\log n)$
- 空间复杂度: $O(1)$

创建堆

- **make-heap(A)**: 从数组A创建堆
- 方法1（插入法）：从一个空堆开始，逐步插入A中的每个元素，直到A中所有元素都被转移到堆中



- 具体算法步骤？

for($i=2$; $i \leq m$; $i++$) insert(A, i);

- 更具体一点？

$len = 1$;

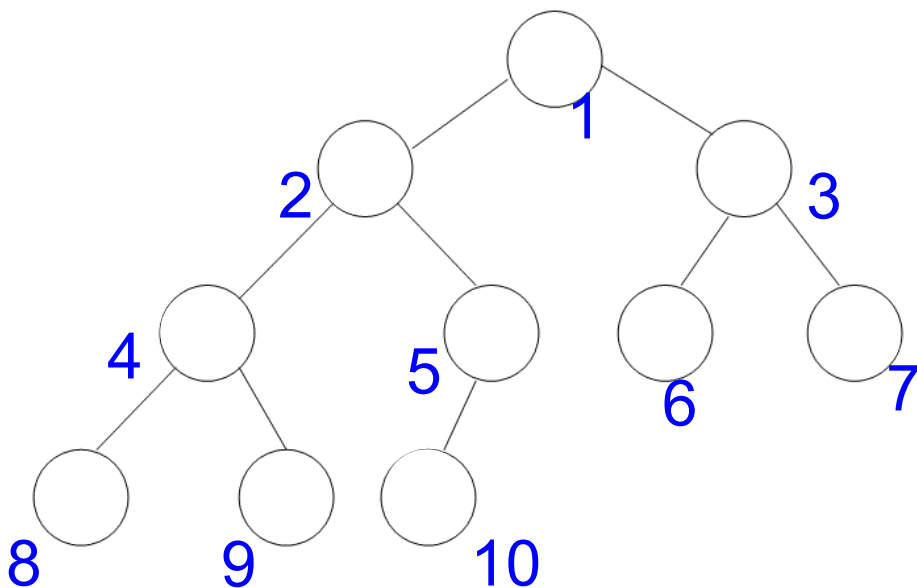
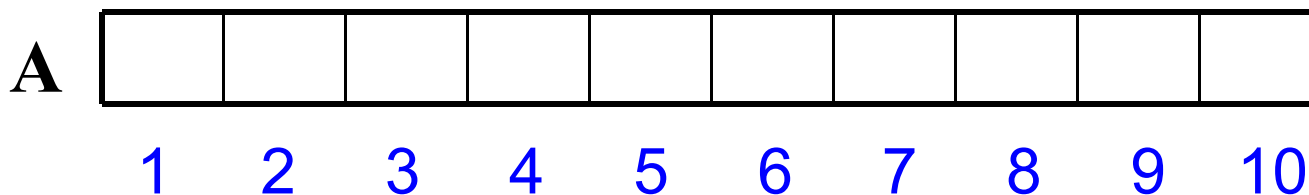
for($i=2$; $i \leq m$; $i++$){ $len++$; Sift-up(A, i);}

创建堆

- `make-heap(A)`: 从数组A创建堆
 - 方法1（插入法）：
 $len = 1;$
 `for( $i=2; i \leq n; i++$ ) {  $len++$ ; Sift-up( $A, i$ ); }`
 - 方法1的时间复杂度为?
 $\log 1 + \log 2 + \cdots + \log(n-1)$
 ➡ $O(n \log n)$
- 有没有更快的方法?

创建堆: 回忆一下怎么判断数组是否是堆的?

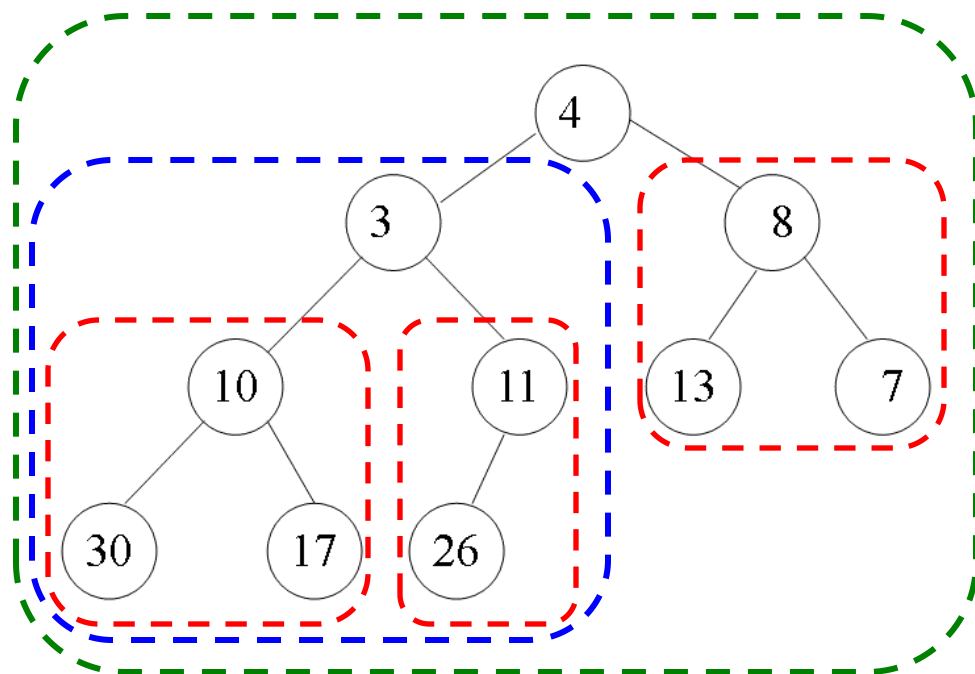
给定数组 $A[1,2,\dots,n]$, 如何判断它是否是堆? 给出算法



创建堆

- $\text{make-heap}(A)$: 从数组A创建堆
- 几乎完全二叉树的任何一棵子树也是几乎完全二叉树

假设数组 $A[1\dots 10]$
 $= \{4, 3, 8, 10, 11,$
 $13, 7, 30, 17, 26\}$



创建堆

➤ make-heap(A): 从数组A创建堆
方法2:

MAKEHEAP (创建堆)

输入: 数组A[1...n]

输出: 将A[1...n]转换成堆

```
{ for ( $i = \lfloor n/2 \rfloor$  downto 1)
    Sift-down(A,i) ; //使以A[i]为根节点的子树
                      //调整成为堆, 故调用down过程
endfor
}
```

创建堆： 举例

24	21	23	22	36	29	30	34	28	27
----	----	----	----	----	----	----	----	----	----

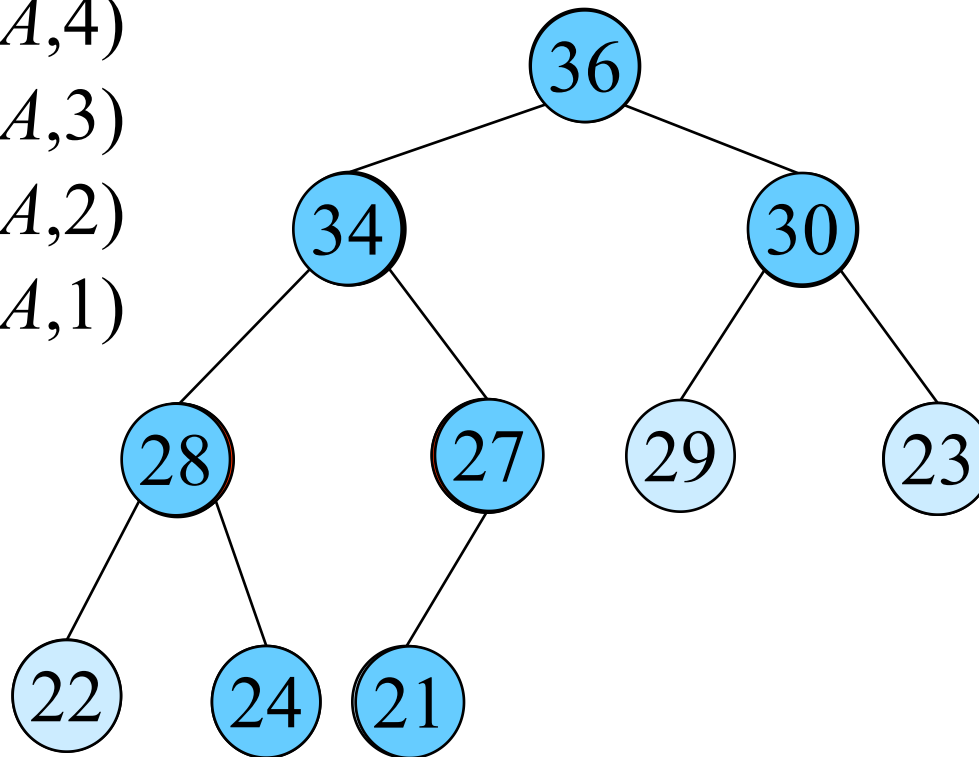
SiftDown($A, \lfloor 10/2 \rfloor = 5$)

SiftDown($A, 4$)

SiftDown($A, 3$)

SiftDown($A, 2$)

SiftDown($A, 1$)



创建堆 (3)

➤ 时间复杂度(假设节点个数为 n):

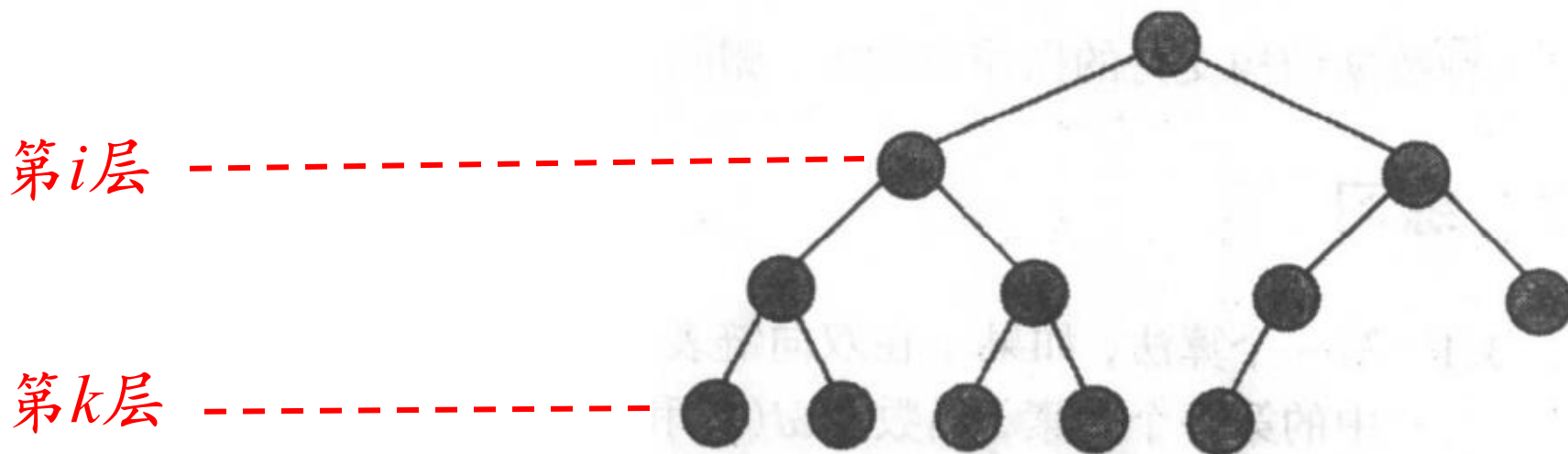
- 树的高度 $k = ?$

$$k = \lfloor \log n \rfloor,$$

- 对于 $0 \leq i < k$, 第 i 层的节点个数?

第 i 层正好 2^i 个节点

- 对于第 i 层的每个节点, down过程最多执行 $k-i$ 次



创建堆 (3)

➤ 时间复杂度 (假设节点个数为 n) :

- 创建堆整个过程, down过程执行次数上限为

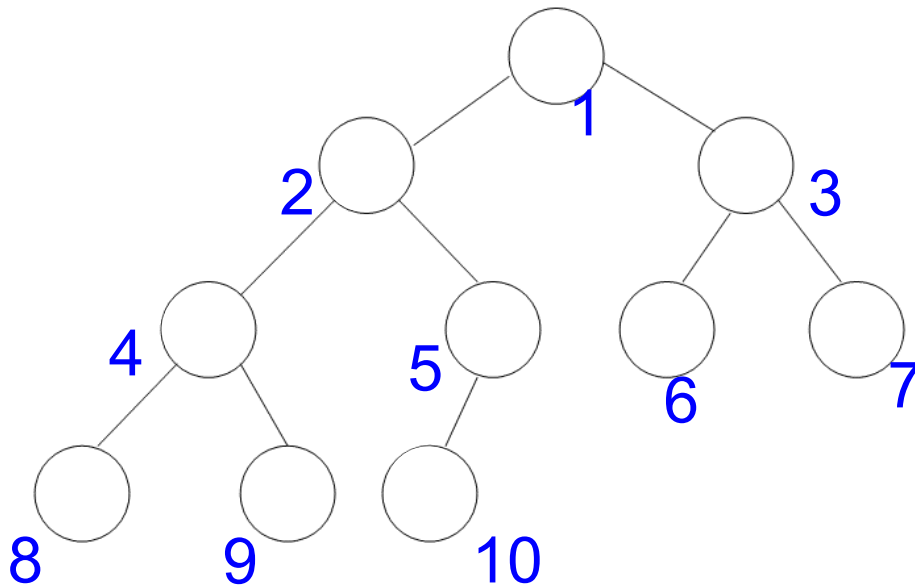
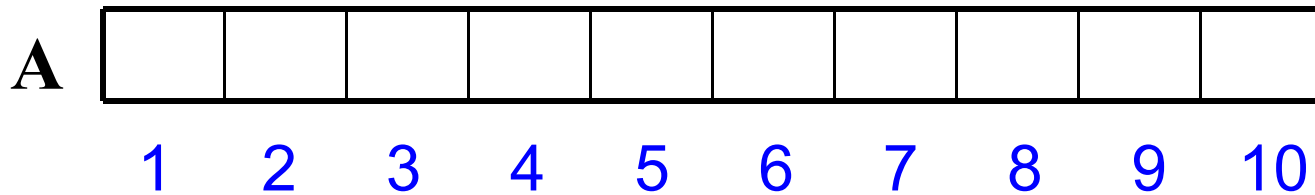
$$\sum_{i=0}^{k-1} (k-i)2^i = \sum_{j=k}^1 j2^{k-j} (\text{令 } k-i=j)$$

$$= 2^k \sum_{j=1}^k j / 2^j = 2^k \cdot \cdot (1)$$

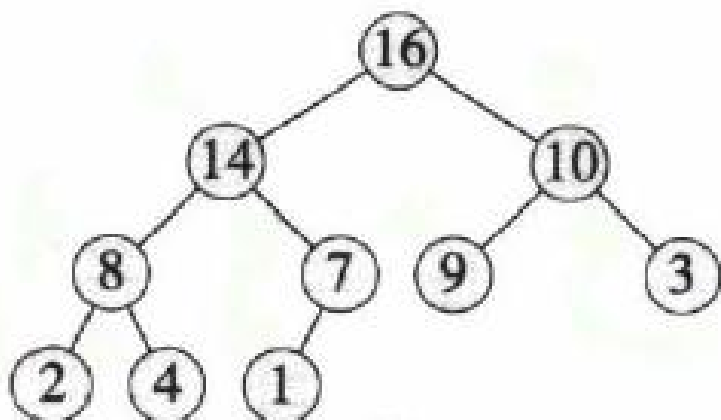
$$\leq n \cdot \cdot (1) < 2n$$

堆排序

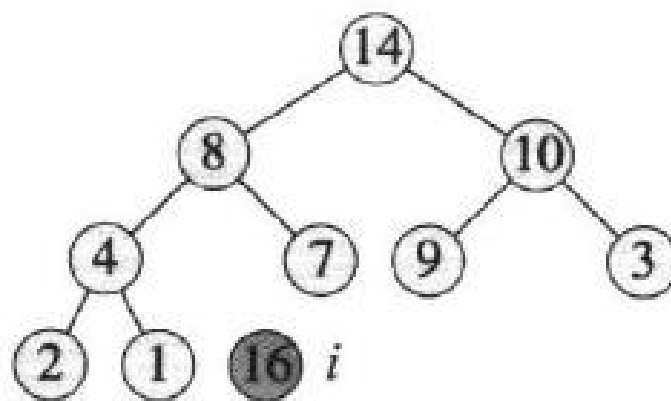
➤ 主要思路



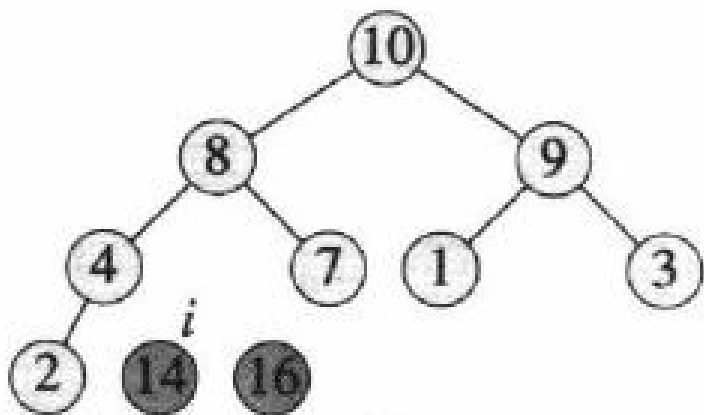
堆排序



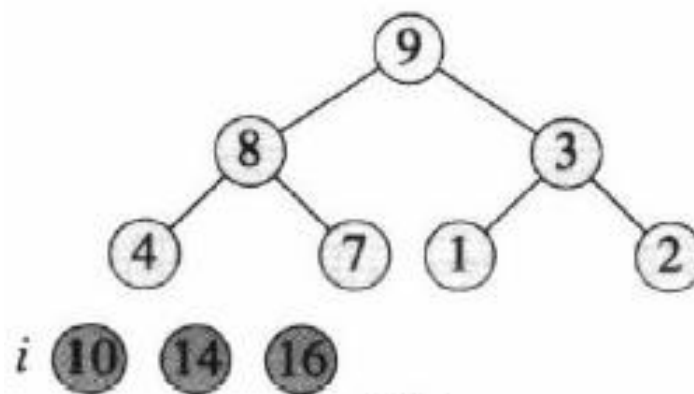
(a)



(b)

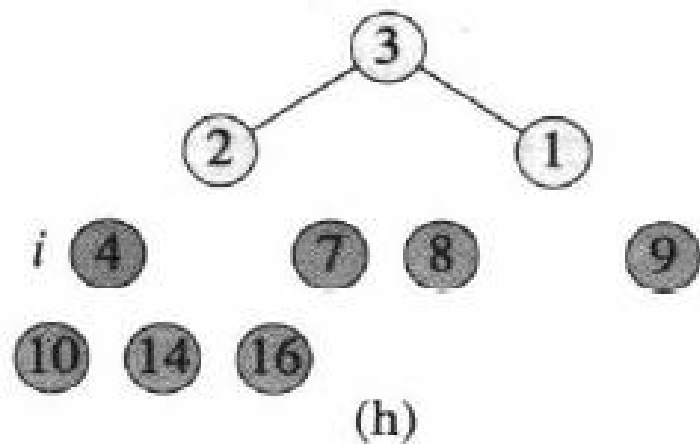
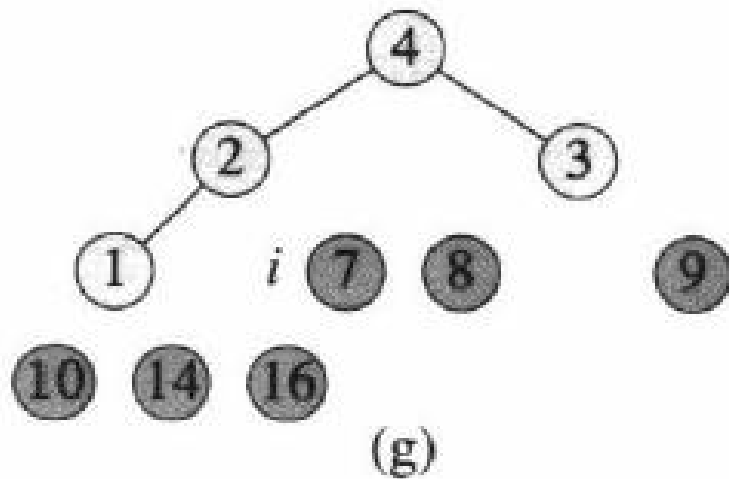
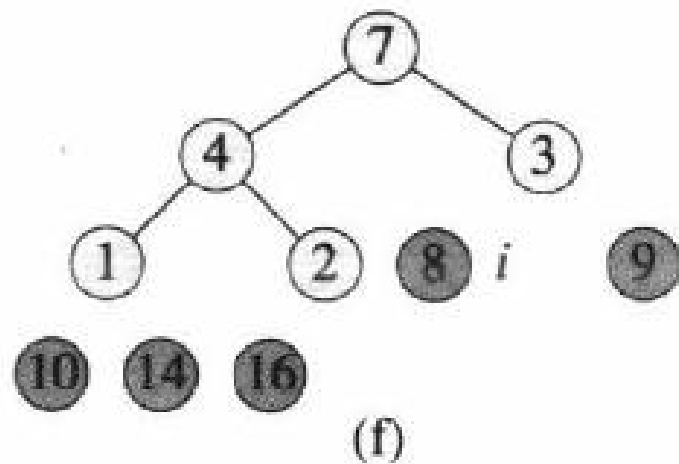
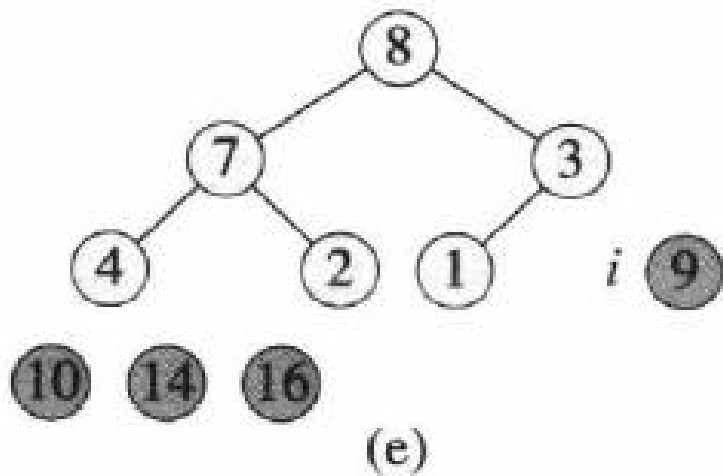


(c)

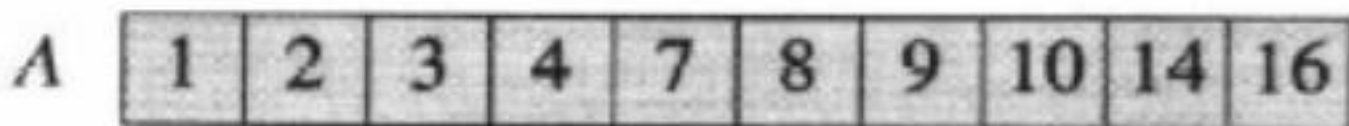
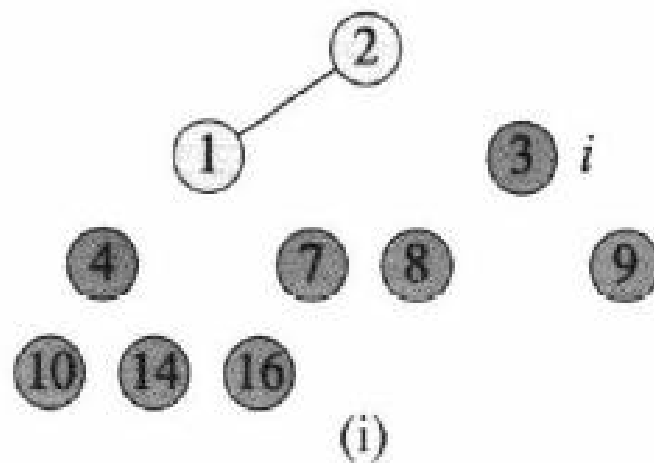
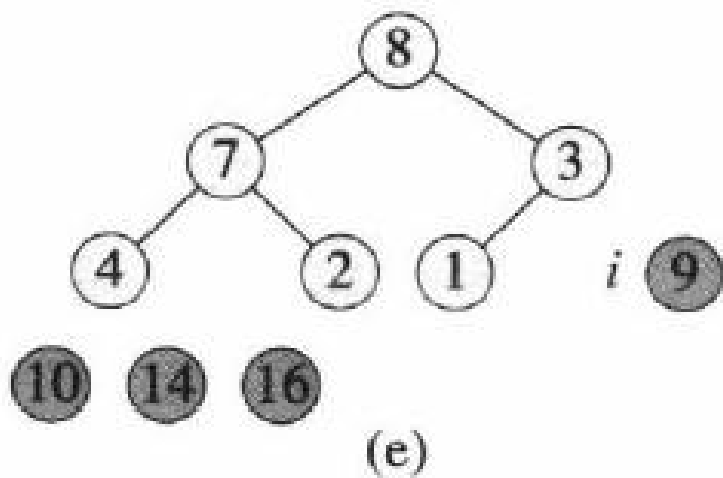


(d)

堆排序



堆排序



(k)

堆排序

算法 HeapSort($A[1..n]$)

输入： n 个元素的数组 $A[1..n]$ 。

输出：以非降序排列的数组 A 。

1. **MAKEHEAP(A)**
2. **for $j \leftarrow n$ downto 2**
3. 互换 $A[1]$ 和 $A[j]$
4. **SIFT-DOWN($A[1..j-1], 1$)**
5. **end for**

堆排序

➤ 空间复杂度: $O(1)$

➤ 时间复杂度

1. **MAKEHEAP**(A)

$O(n)$

2. **for** $j \leftarrow n$ **downto** 2

3. 互换 $A[1]$ 和 $A[j]$

4. **SIFT-DOWN**($A[1 \cdots j-1], 1$)

$O(n \log n)$

5. **end for**

➤ 时间复杂度: $O(n \log n)$

堆的应用

- 问题：海量实数中（设为 n 个， n 为一亿级别以上）找到最小的 k 个（一万级别以下）的数集合）
- 你有什么解决思路？时间复杂度是多少？

堆的应用

- 问题：海量实数中（设为 n 个， n 为一亿级别以上）找到最小的 k 个（一万级别以下的数集合），即 $n \gg k$.
- 你有什么解决思路？
 - ✓ 直接排序： $O(n \log n)$
 - ✓ 每次找最小的： $O(n * k)$
 - ✓ k 个元素建大顶堆： $O(n \log k)$

算法	时间复杂度	空间复杂度
插入排序	$O(n^2)$	$O(1)$
自底向上合并排序	$O(n * \log n)$	$O(n)$
选择排序	$O(n^2)$	$O(1)$
冒泡排序	$O(n^2)$	$O(1)$
堆排序	$O(n * \log n)$	$O(1)$

- 第二次作业：随机生成一千、一万、五万、十万个数，实现上述每个算法，进行测试分析，给出实验报告（包括5部分：实验目的、方法、数据集和系统配置、结果与分析、感想）
- 提交时间：4月3日课前（可以交打印的）